Co-Design of Morphing Winged Drones and Hebbian Plastic Controllers

Student: Salvatore

Supervisors: Andrea, Fuda

The **issue:**
In aerial robotics, morphology and control are typically optimized separately. However, morphology directly shapes system dynamics, controllability, and the structure of sensory feedback. Designing body and control independently prevents systematic understanding of their interaction.
The **challenge:**
In morphology–control co-design, learning-based controllers must often be retrained from scratch for each candidate morphology. This makes evolutionary search computationally prohibitive.
The **Contribution:**
We propose a co-design framework that separates three roles across timescales:

- Pretraining to learn general flight behavior across many morphologies.
- Evolution to optimize morphology and adaptation parameters.
- Online Hebbian plasticity to enable rapid specialization during evaluation.

This structure reduces retraining cost while allowing controlled analysis of how structure, prior learning, and lifetime adaptation interact.

1 Project motivation

The design of aerial robots traditionally separates morphology and control: a body is engineered first, and a controller is later optimized for that fixed structure. However, morphology directly shapes system dynamics, controllability, and the structure of sensory feedback. For this reason, body and control should not be treated as independent problems.

Morphology–control co-design aims to jointly optimize structure and control policy. Learning-based controllers are particularly suitable in this setting, as they can handle complex and nonlinear flight dynamics. However, they introduce a major limitation: computational cost. Training a new controller from scratch for every candidate morphology makes large-scale evolutionary search impractical.

The central idea of this project is to concentrate learning in an initial pretraining phase and avoid repeated full retraining during evolution.

First, a general control policy is trained across a wide distribution of drone morphologies using reinforcement learning (PPO). The objective is not to specialize to a single body, but to capture morphology-agnostic regularities in flight behavior.

An evolutionary outer loop is then introduced to search over morphologies. Crucially, evolution can also optimize the parameters governing online adaptation, including initialization of adaptive components and coefficients of Hebbian plasticity.

During evaluation of each candidate morphology, selected parts of the controller adapt online through Hebbian updates.

This enables rapid specialization to morphology-specific dynamics while keeping computational cost low.

2 Project work packages

WP1 – Evolving Rules for a specialized single-drone controller

Objective: Evolve Hebbian Rules for the head of a Reinforcement learning pre-trained controller

Method:

- Implement Hebbian rules on top of the head of a controller.
- Evolve Rules using CMA-ES using the same fitness function as RL.
- Optimize for robust improved performance.

Outcome:

- A set of rules improving the performance of a Reinforcement Learning controller.

WP2 – Evolutionary Co-Design of rules and morphology

Objective: perform joint optimization of morphology and adaptation mechanisms without repeated controller retraining.

Method:

- Introduce an evolutionary NSGA-II outer loop for morphology.
- Evolve morphology parameters every N rules evolution iterations.
- Allow the overall evolution to optimize:
  - initialization of adaptive (plastic) parameters,
  - potentially wheights of the control network,
  - rules governing Hebbian plasticity.
  - parameters dictating morphology

Outcome:

- Efficient co-design where structure and adaptation mechanisms are shaped together.

WP3 – Meta-learned representations (ANIL extension)

Objective: compare standard generalist pretraining with meta-learned representations.

Method:

- Learn a generalist feature extractor (ANIL).
- Enable Hebbian plasticity after feature extractor during evaluation.

Outcome:

- Analysis of how meta-learned structure interacts with Hebbian plasticity and evolutionary optimization.

3 Experiment Organization and Results

3.1 Research Questions

1. *Can Hebbian learning improve a general PPO policy?*
2. *Can Hebbian co-design beat plain co-design?* (Together with morphology, evolve last-layer weights directly vs. evolving per-weight ABCD rules that update them online.)
3. *What morphological traits persist across many evolution runs?*
4. *What is the impact of plasticity in the head weights when changing morphology?* (CCA analysis between baseline and adapted controllers.)
5. *What is the recovery speed of the plastic controller with respect to the deviation from non-dominated morphologies?*
6. *Can Hebbian plasticity rescue performance under in-flight "accidents" (frozen weights, broken wing)?*

3.2 Pipeline Overview

The project is structured as three nested optimization loops at progressively slower timescales:

1. **Pretraining (WP1, inner-most in timescale but largest one-shot cost).** A single PPO controller is trained across a wide pool of randomly sampled drone bodies. The actor never sees the body it is currently flying; only the critic is allowed access to body parameters. This produces a morphology-blind generalist controller that is reused as a frozen starting point for everything downstream.
2. **Rule evolution (WP2 inner loop).** For a given body (or small pool of bodies), an evolution strategy searches per-synapse plasticity rules that modulate the controller's last layer online during a flight. The underlying pretrained controller is never gradient-updated; its weights are restored at the start of every rollout, and the rule alone is responsible for any specialization observed during that rollout.

3. **Morphology evolution (WP2 outer loop).** A multi-objective evolutionary algorithm searches the morphology space; each candidate body is scored by running the rule-evolution inner loop on it and reading off the best fitness achieved.

Every run produces a reproducible bundle (frozen config, random seeds, base controller checkpoint, environment description) and a timestamped output folder so any reported number can be reproduced from the artifacts on disk.

3.3 Work Package 1 — Foundation Policy Training

3.3.1 Goal

WP1 produces a *single* controller that can fly a wide distribution of winged drones through forests. The training distribution is a pool of randomly generated drone bodies, each described by a 15-dimensional morphology genome (wing span, chord, sweep, twist, airfoil shape, etc.). At every training step many parallel environments are simulated, each containing one drone body from the pool. The controller receives the same observation regardless of which body it is currently flying — only the critic is given the body's parameters — so the resulting policy is forced to extract morphology-agnostic regularities.

The actor is never given the morphology; the critic is. This asymmetry is the design choice that makes downstream Hebbian adaptation interesting: the actor is by construction a generalist, and any morphology-specific specialization at evaluation time has to come from the plasticity rule, not from new observations.

3.3.2 Reward function

The per-step reward used during training is:

$$r_t = \Delta t \cdot 50 \sum_i w_i \, f_i(s_t, a_t)$$

with the following sub-terms:

- **Progress** ($w = +0.5$): Gaussian reward for tracking the commanded forward speed v^*:

$$f_{\text{progress}} = \exp\left(-\frac{1}{2}\left(\frac{v_x/v^* - 1}{0.25}\right)^2\right)$$

  where v_x is the drone's forward velocity projected onto the x-axis.

- **Smoothness** ($w = -0.1$): Penalty for large changes in the action between consecutive timesteps:

$$f_{\text{smooth}} = \sum_j \left(a_j^t - a_j^{t-1}\right)^2$$

- **Obstacle** ($w = -0.1$): Penalty for proximity to obstacles, computed from depth rays using an anisotropic

distance metric:

$$f_{\text{obstacle}} = \sum_k \exp\bigl(-1.5 \cdot \bar{d}_k\bigr), \qquad \bar{d}_k = \frac{\sqrt{x_k^2 + 24\, y_k^2}}{5} \ \text{(clipped at 0.5)}$$

where (x_k, y_k) are the body-frame projected coordinates of the k-th depth ray, and lateral distance is weighted $24\times$ more than forward distance.

- **Cosmetic** ($w = -1.0$): Penalty for asymmetric wing servo usage:

$$f_{\text{cosmetic}} = (q_0 + q_1)^2 + (q_2 - q_3)^2 + q_5^2$$

where q_j are the servo joint positions.

- **Crash** ($w = -10.0$): One-shot terminal penalty applied when the drone crashes (ground contact $z < 0.1\,\text{m}$, lateral out-of-bounds, or contact collision):

$$f_{\text{crash}} = \frac{\mathbf{1}[\text{crash}]}{\Delta t \cdot 100}$$

- **Angular velocity** ($w = -0.005$): Penalty for large angular velocity:

$$f_{\text{angular}} = \|\boldsymbol{\omega}\|_2$$

- **Height** ($w = -0.003$): Penalty for flying below the target altitude $z^* = 10\,\text{m}$ (one-sided):

$$f_{\text{height}} = \bigl(z - z^*\bigr)^2 \cdot \mathbf{1}[z < z^*]$$

- **Energy** ($w = -0.001$): Penalty proportional to instantaneous motor power consumption P:

$$f_{\text{energy}} = P$$

3.3.3 Policy architecture and observation space

The controller is a recurrent policy. Observations are first processed by an LSTM of hidden size 128. The LSTM output then passes through a two-layer feed-forward network with widths 128 and 32 and ELU activations, and finally through a linear layer that maps the 32-dimensional pre-output features to the 7 actions, squashed with $\tanh$ and rescaled to the actuator ranges. The critic mirrors this structure with its own independent LSTM of size 128 followed by a wider feed-forward network (two layers of 128).

The actor and critic having *independent* recurrent memories is a deliberate departure from the standard recurrent actor-critic pattern, which shares a single recurrent module. With a shared memory the critic's privileged information about the body parameters leaked into the actor's latent state through the shared recurrence, and the actor stopped generalizing. Decoupling the memories was a precondition for stable wide-distribution training.

The 36-dimensional observation given to the actor contains: normalized altitude, body orientation, linear velocity, twenty obstacle-distance sectors from a depth sensor, the previous action, and the commanded forward speed. The 7

actions are: throttle, two wing-sweep angles (left/right), two wing-twist angles (left/right), elevator, and rudder.

3.3.4 Experiment WP1.A — How wide does the training distribution need to be?

Question. The actor never sees which body it is flying. Does a single PPO controller actually generalize across a wide pool of bodies, or does it collapse to an average policy that flies none of them well?

Setup. Sweep the pool size between a single body, around one hundred, two hundred, and over two hundred bodies, and evaluate every resulting controller on around five hundred fresh bodies (most unseen during training), in twenty random forests each. The single-body run is kept as the specialist upper bound.

Finding. A morphology-blind controller trained on a pool of around one hundred to a few hundred random bodies flies most unseen bodies sampled from the same distribution. Extreme bodies (outliers in span, sweep, or airfoil camber) consistently underperform — and this performance dip is the gap that motivates the second work package. The single-body specialist, predictably, is the best on its own body but generalizes poorly. Wide-pool training was only possible after the actor and critic recurrent memories were separated, as described above.

3.3.5 Experiment WP1.B — Last-layer width vs. search-space size

Question. The downstream rule-evolution stage operates on the controller's last linear layer; its search space grows in proportion to the width of the last hidden layer. A narrow last layer makes the evolutionary search tractable; a wide one gives the PPO controller more capacity. Where is the trade-off?

Setup. Keep the recurrent state size fixed at 128 and keep the critic wide (128×128). Vary the actor's last hidden layer between width 32 and width 64, all other settings equal, and train on a wide-pool catalog.

Finding. A last-hidden-layer width of 32 is the smallest tested where PPO still converges on the wide pool without measurably degrading flight performance relative to width 64. It nearly halves the size of the downstream evolutionary search (a width of 64 would give $4 \times 64 \times 7 = 1792$ parameters to evolve, against $4 \times 32 \times 7 = 896$ for width 32). The wider critic is kept because the privileged information it consumes benefits from extra capacity, and critic width does not enter the downstream search at all.

3.3.6 Experiment WP1.C — Privileged critic observations

Question. Asymmetric actor-critic methods often help when the critic can see information that is hidden from the actor. Which privileged signals matter for our setting?

Setup. Selectively expose, to the critic only, four extra observations on top of the genome: angular velocity, joint positions, joint velocities, and a feedback estimate of the actually-delivered thrust. The actor's observation is held fixed at 36 dimensions throughout. The toggles are wired so each can be turned on or off independently.

Finding. Exposing joint position/velocity and angular velocity to the critic tightens the value estimate. The improve-

ment is most pronounced on difficult bodies, where the actor's depth-and-velocity observation alone is too coarse for the critic to reliably score recoveries. The thrust-feedback signal is borderline but is kept because it is essentially free. The toggles are preserved for downstream ablations that ask whether the plastic rule still helps when the critic was trained without privileged inputs.

3.3.7 Experiment WP1.D — Forest curriculum vs. randomization

Question. Should training forests be fixed-density, randomly drawn per episode, or grow from open at the start to dense at the end within a single episode?

Setup. Three regimes:

1. Fixed forest density across the run.
2. Per-episode random density (the initial density of each forest is drawn from a range at episode reset).
3. Within-episode densification: trees are sparser at the start of the corridor and denser at the end, so every episode begins in open space and gradually becomes harder.

The three regimes can be combined.

Finding. Combining the per-episode random initial density with the within-episode gradient produces a controller that handles both open and dense corridors without retraining. Fixed high density collapsed training (most of the early population crashed before the policy could learn obstacle avoidance, leaving no gradient to follow). Fixed low density produced controllers that could not slalom. The combined curriculum is therefore the training default. A complementary change — randomizing the drone's initial attitude (not just position and velocity) at every reset — is also kept; this matters once the outer loop later generates bodies whose passive trim is far from level flight.

3.3.8 Experiment WP1.E — Parallel training infrastructure (a graveyard)

Question. Can PPO training be sped up by splitting work across processes or GPUs?

Tries.

1. *Multi-GPU dispatcher.* One worker per GPU pushing rollouts to a central PPO update. Initially crashed reliably around iteration three hundred from a slow memory leak: an autograd graph was being silently built through the simulator inside each worker, and the GPU memory it pinned could not be freed. Once the leak was patched, training ran cleanly — but the throughput gain over a single well-tuned GPU was small, because the dominant cost was the simulator step itself, not the policy update.
2. *Many independent scenes across worker processes.* Instead of one big simulator state, run many smaller simulator states in parallel, each in its own process. A systematic sweep over (number of bodies per scene, number of parallel scenes, number of environments per scene) was run on the cluster to map the throughput-vs-memory

Pareto frontier. Parallel scenes did help on the first run — when the simulator's compilation cache was empty, parallel processes hid the compilation latency — but as soon as the cache warmed up the simpler single-process backend was within roughly ten percent of the best parallel configuration, and the parallel stack was significantly more brittle (inter-process tensor transfer, serialization, per-process simulator initialization quirks). The parallel stack was eventually removed.

3. *Sharded body pool.* A variant in which each worker process owned a subset of the body pool and rotated bodies through a single simulator state to bound VRAM. Useful in principle for very large pools, but the pool never grew to a size where VRAM was the binding constraint, and the rotation logic was a steady source of subtle bugs (lingering simulator state surviving the body swap). Abandoned.

What survived. A single, mode-selecting training entry point: it routes to single-body, multi-body, or parallel-scene mode based on the configuration alone. The conclusions of the parallelization sweep (typical recipe: low hundreds of bodies, around sixty environments per body, single GPU, two thousand PPO iterations) were locked in as the production defaults rather than rederived on every cluster submission. The overall lesson is that parallelization paid less than expected because simulator compilation amortizes quickly across runs and the dominant cost is GPU simulator throughput, not host-side orchestration.

3.3.9 Experiment WP1.F — Drone-mass drift and aerodynamic-force clipping

Issue. Mid-project the morphology decoder was refactored and the on-disk description of the canonical drone silently changed mass (from roughly $0.75\,\mathrm{kg}$ to roughly $0.60\,\mathrm{kg}$, a $\sim 19\%$ change). Controllers trained against the old mass are not interchangeable with controllers trained against the new one — the feed-forward throttle that PPO converges on is morphology-specific.

Resolution. Both training and evaluation now regenerate the canonical drone description from the genome on the fly, instead of relying on a stale file. Controllers trained before the refactor were retrained from scratch. Separately, the aerodynamic-force clipping threshold was found to be too aggressive on outlier bodies (large-span, high-camber airfoils): forces were being silently clipped, distorting the policy on those bodies. The threshold was raised.

3.3.10 Robustness evaluation protocol

The final controller is evaluated on around five hundred bodies sampled fresh from the morphology distribution (most never seen during training), in twenty random forests each. Reported metrics are: success rate (the drone reaches the end of the corridor), crash rate, mean progress along the corridor, mean error in tracking the commanded forward speed, and mean cost of transport. Because the actor is morphology-blind, success on outlier bodies is expected to dip. That dip is exactly the gap the second work package targets.

3.4 Work Package 2 — Inner Loop: Evolving Hebbian Rules

3.4.1 Goal

With the generalist controller frozen, WP2 searches for *plasticity rules* that update the controller's last linear layer online, during a flight. The controller's pre-trained weights are restored at the start of every rollout, so the rule is solely responsible for any morphology-specific specialization observed during that rollout.

3.4.2 Plasticity rule

For each weight w_{ij} in the last linear layer, with x_i the corresponding pre-output feature and y_j the post-tanh action, the plasticity rule has four evolved per-synapse coefficients $A_{ij}, B_{ij}, C_{ij}, D_{ij}$ and produces a weight update of the form

$$\Delta w_{ij} \;=\; \eta_{ij} \cdot k_{ij} \cdot \big(A_{ij}\, x_i y_j + B_{ij}\, x_i + C_{ij}\, y_j + D_{ij}\big).$$

The weight is then updated and softly anchored toward its frozen pre-trained value $w_{ij}^{(0)}$:

$$w_{ij}^{(t+1)} \;=\; \mathrm{clip}\Big((1-\lambda_{ij})\, w_{ij}^{(t)} + \lambda_{ij}\, w_{ij}^{(0)} + \Delta w_{ij},\; -w_{\max}, +w_{\max}\Big).$$

The anchor term, controlled by per-synapse λ_{ij}, pulls drifting weights back toward the pre-trained baseline rather than toward zero. An optional Oja-style coefficient

$$k_{ij} \;=\; 1 \;-\; \frac{y_j^2\,\big(w_{ij}^{(t)} - w_{ij}^{(0)}\big)}{x_i\, y_j + \epsilon}$$

shrinks the update proportionally to how far the weight has drifted; when this coefficient is disabled, $k_{ij} \equiv 1$ and the rule reduces to a standard ABCD update with anchor and decay.

Biases are never modified. Weights are reset to the pre-trained values at the start of every rollout *and* at the start of every generation, so the rule alone explains every behavioral difference observed within a rollout.

The evolved parameters are the four coefficients per synapse (giving $4 \times 32 \times 7 = 896$ genes for the production controller width), plus optionally per-synapse λ and η when those are also subject to selection.

3.4.3 Experiment WP2.A — Joint evolution of rules and morphology in one flat genome (failed)

Question. Can a single multi-objective evolutionary algorithm jointly evolve plasticity rules and morphology in one large genome, with rule genes and morphology genes side by side?

Setup. An initial implementation concatenated the rule genome and the morphology genome into a single high-dimensional vector ($\sim$ 1800 entries dominated by rule coefficients) and ran a multi-objective evolutionary search against a Pareto front of velocity, energy efficiency, and progress, with optional smoothness, crash rate, and adaptability objectives. Five preset configurations covered the ablation grid: rules-on/morphology-on, rules-on/morphology-off,

rules-off/morphology-on, both-off (a baseline), and rules-and-morphology-on with crossover disabled (mutation only).

Why it failed. Three causes compounded.

1. The multi-objective evolutionary algorithm used — with polynomial mutation and simulated-binary crossover — is a poor fit for a smooth ~ 1800-dimensional continuous problem. On the rule subspace alone, convergence was orders of magnitude slower than what a covariance-adapting evolution strategy achieves.
2. Crossover on a high-dimensional continuous rule vector was actively harmful: rule coefficients interact non-linearly inside the controller, so a crossover-mixed offspring frequently performed worse than either parent.
3. Coupling rules and morphology into one population blended two very different selection signals: the rule's goal is to maximize cumulative reward on a *fixed* body, while the morphology's goal is to find Pareto-optimal bodies. Generations that improved one frequently degraded the other.

Resolution. The flat-genome co-design was discarded. Morphology evolution was reintroduced separately, as a slower outer loop *wrapping* the inner rule-evolution loop (see Experiment WP2-Outer.A).

3.4.4 Experiment WP2.B — Rules-only evolution with a covariance-adapting strategy

Question. With morphology held fixed, can a covariance-adapting evolution strategy (CMA-ES) find rules that improve over the frozen generalist controller, using the same scalar reward used to train the controller?

Setup. Each individual is the 896-dimensional rule vector (optionally extended with per-synapse λ and η). Population size around one hundred, initial step size roughly 7.5% of the genome range. The fitness is the sum of the per-step reward over the rollout — identical to what was optimized during pre-training, which keeps the comparison apples-to-apples. Two initialization variants were compared:

- *Zero initialization.* Starting all four rule coefficients at zero, the first generation is behaviorally indistinguishable from the frozen controller. Any subsequent improvement is unambiguously attributable to the rule.
- *Random initialization.* Starting each rule coefficient uniformly inside a small range. Faster initial exploration but destructive early rules.

Finding. Zero initialization is the production choice. Random initialization typically loses to zero initialization for the first several generations because most random rules destroy the controller; on the compute budgets we use, zero initialization wins on net. Equally important, zero initialization makes the inner-loop fitness curve trivially interpretable — generation zero *is* the baseline.

3.4.5 Experiment WP2.C — Stabilizing the plasticity rule

Question. Plain ABCD updates can drive weights unboundedly when the optimizer finds aggressive coefficients, eventually destroying the frozen controller. What regularizer keeps the rule bounded *without* flattening the rule space the optimizer is searching?

Approaches. Two modifications, both toggleable so they can be ablated:

- **Oja-style coefficient.** Multiplies into the update and shrinks it when the current weight has drifted far from its pre-trained value, as in the form given above.
- **Anchored decay.** The standard interpretation of weight decay pulls weights toward zero, which here destroys the pre-trained controller. The decay term used in the rule above instead pulls each weight toward its frozen pre-trained value. This regularizes drift back toward the known-good controller rather than toward a trivial collapsed solution.

Finding. The anchored decay is kept in production. Without it, runs that found aggressive rules typically learned to fly briefly and then unlearned within a few generations as weights drifted. With the anchor present, the optimizer is free to set λ small (or zero) on synapses where weight changes are productive, and large on synapses where they need taming. The Oja coefficient is currently disabled in production: empirically the anchor alone is sufficient, and the Oja coefficient sometimes over-regularized small productive updates, but the toggle is preserved as an ablation knob.

3.4.6 Experiment WP2.D — How to evaluate one individual on many bodies

Question. Each individual must be evaluated on multiple bodies, otherwise the rule overfits to whatever single body it sees. What is the fastest way to run many different bodies in parallel inside the simulator?

Approaches tried.

1. *One simulator state holding many drones at once.* Pack many different bodies into a single simulator scene and step them together. On paper, maximum GPU parallelism. *In practice*, an aerodynamic numerical failure on any one body silently propagated through the shared internal simulator state, corrupting every body's next step. The mode was abandoned for inner-loop evaluation.
2. *One independent simulator state per body, stepped sequentially.* Slower in theory because the loop runs on the host, but the per-body simulator step dominates anyway. *Robust against per-body numerical failures* — a crash in one body does not touch any other.
3. *Independent simulator states distributed across worker processes.* Useful when a single simulator state does not saturate the GPU; spreads bodies across workers in a round-robin fashion.

Finding. The independent-states approach (sequential or worker-distributed) is the production choice. Worker distribution typically saturates a single GPU at two workers when the per-state compilation cost is small; multi-GPU sharding was added later to scale beyond one card.

Side experiment that didn't pay. Skipping the simulator step for environments that had already terminated — naively appealing as an optimization — was tried and reverted the same day. The cost of branching to mask done environments exceeded the savings, because a terminated environment is already cheap to step (zero throttle, no aerodynamic contribution). The general lesson is that "obvious" optimizations on top of a bulk-parallel simulator usually lose to the simulator's own throughput.

3.4.7 Experiment WP2.E — Inner-loop fairness

Question. For an evolutionary search to rank individuals correctly, any two individuals must be compared on the *same* task. What sources of variance had to be eliminated?

Adjustments.

- *Per-individual forest layouts.* The first evaluator drew a fresh forest for every individual, so individuals were not ranked on identical tasks. The fix is to generate a fixed set of forests once per generation and reuse them for every individual within that generation.
- *Per-individual commanded forward speed.* Same issue. Commanded speeds are now drawn once per generation as a fixed linearly-spaced ladder, reused across individuals.
- *Deterministic vs. stochastic action sampling.* The pre-trained controller was trained with stochastic action sampling, but the evaluation wrapper initially evaluated it deterministically. The mismatch silently degraded the baseline measurement until the stochastic flag was wired through. (Stochastic sampling itself is also a per-individual variance source, so the noise-ablation framework, below, treats it as a controlled noise channel that can be switched off in isolation.)
- *Forest difficulty drift across generations.* Even with forests fixed within a generation, different generations face different forests, so a late-generation fitness improvement could be due to easier forests rather than better rules. The fix is to periodically re-evaluate the frozen baseline controller (with no rule) on the current generation's forests; the population's fitness is then reported normalized against this moving baseline.

3.4.8 Experiment WP2.F — Specialist upper bound

Question. A plasticity rule that adapts online cannot reasonably be expected to beat a controller specifically trained from scratch on that body. How much of the gap between the generalist controller and a body-specific specialist can the rule actually close?

Setup. Alongside the generalist baseline (frozen controller, no rule), each generation also evaluates a specialist reference: the frozen controller run on each body independently with no rule. The specialist gives a per-body reference for what the underlying controller can do on that body without any adaptation. The plasticity rule's value is then expressed as the fraction of the generalist-to-specialist gap that the rule closes.

Finding. The "fraction of specialist gap closed" metric is much more informative than absolute reward improvement, because forest difficulty fluctuates and the absolute reward scale varies by body. The same metric is reported per reward component (progress, velocity tracking, energy, smoothness) so we can see *which* aspects of specialization the rule actually recovers.

3.4.9 Experiment WP2.G — Noise ablations

Question. The simulator has many independent noise channels — action latency, mass shift, center-of-mass shift, joint command bias, joint command step noise, aerodynamic noise, and stochastic action sampling. Which of these is the plasticity rule actually compensating for? Would its improvement vanish if we made the simulator deterministic?

Setup. Each noise channel is independently toggleable at evaluation time — including channels that were enabled during the pre-training of the controller. So a rule trained against noisy actuator latency, say, can be evaluated against a perfectly clean actuator, to ask whether the rule's gain came from latency compensation specifically.

Why it matters. "The rule helps because it compensates for action latency" and "the rule helps because it adapts to morphology asymmetry" are scientifically very different claims, and only this kind of decoupled toggle lets the two be told apart.

3.4.10 Experiment WP2.H — Densification curriculum in the inner loop

Question. If the inner loop evaluates rules on a fixed forest density, the optimizer finds rules that just survive that density. Can a curriculum push the rules to also handle harder forests?

Setup. The inner loop can apply a per-generation slope to the forest's initial density: generations start in open corridors and gradually densify as the population converges, so early generations are not overwhelmed before any rule has time to form.

Finding. The curriculum is the production default for runs targeting hard forests. Without it, runs at a high fixed density collapsed in the first couple of generations — most individuals crashed immediately and the optimizer had no fitness gradient to follow. With the curriculum, the population reliably reaches the target density by the end of the run. The corridor itself was also lengthened to roughly twice its original length, because post-rule the controller frequently flew the original corridor in a single rollout, leaving no signal in the tail of the episode.

3.4.11 Experiment WP2.I — Body refresh period

Question. If the inner loop evaluates each individual on a fixed small pool of bodies for the whole run, the optimizer may overfit rules to that pool. If we refresh the body pool too often, the optimizer never gets a stable fitness signal. What is the right refresh period?

Setup. A sweep over three refresh periods (every two, four, or eight inner generations), each repeated with two parallel evaluator widths to control for any worker-count interaction, all other settings held identical. Each configuration runs the same number of inner generations, the same population size, the same pre-trained controller, and the same noise/density configuration.

Expected output. For each refresh period, two quantities are reported: (i) the fitness gap, relative to the specialist reference, on the bodies the run actually trained on (in-pool); and (ii) the same gap measured on a held-out body pool sampled fresh at evaluation time. The held-out measurement is the actual generalization metric; the in-pool

measurement only diagnoses overfitting. The optimal refresh period is the one that maximizes (ii) without collapsing (i).

3.4.12 Experiment WP2.J — Accident recovery

Question. A morphology-blind generalist controller will fail outright when a wing is structurally damaged or some control pathway is suddenly disabled. Does the plasticity rule rescue performance in that regime?

Setup. Two "accident" scenarios injected into a rollout at a chosen timestep:

- **Broken wing.** Aerodynamic forces on one wing are zeroed for the rest of the rollout, mimicking structural damage and giving the drone a sudden left/right asymmetry the underlying controller never trained against.
- **Frozen plasticity.** Plastic weight updates are switched off at a chosen timestep. The controller continues to fly with whatever weight configuration the rule had produced at that moment; this isolates *learning during the rollout* from *the steady-state weight configuration* the rule converged to.

The two scenarios can be combined for the strongest stress test: the drone is asymmetric *and* the rule can no longer adapt to that asymmetry after the event.

Expected output. For each body, three rollout populations are compared — baseline (no rule), plasticity rule with online updates, plasticity rule frozen at the accident time — on time-to-crash and progress past the accident event. If the online-updating rule flies meaningfully further than both the baseline *and* the frozen-rule version, then the rule is providing genuine online recovery and not just a better static last-layer configuration. Visualizations of the weight changes before and after the event let us read off which weights the rule moves in response to the asymmetry.

3.4.13 Experiment WP2.K — Structured forests as cleaner stress tests

Question. Random forests are noisy; rule comparisons across runs are blurred by forest-to-forest difficulty variation. Can a structured forest layout give a cleaner comparison?

Setup. A growing lattice forest: trees placed on a regular lattice with the lattice spacing tightening as the drone progresses along the corridor (so density grows deterministically within an episode). The layout is fully reproducible from a single seed and offers a clean per-position difficulty curve that does not depend on Poisson clustering.

Finding so far. The lattice forest is used as a controlled stress test in evaluation rather than as a training distribution. Rankings between rule variants are tighter on the lattice than on random forests, because the per-position difficulty is no longer a confound.

3.4.14 Experiment WP2.L — Latent-space analysis (ongoing)

Question. What does the plasticity rule actually change in the controller's internal representation? Does an adapted controller occupy a different region of the last hidden layer's activation space than the baseline?

Setup. Last-hidden-layer activations are recorded over a full rollout (for both the baseline and the plasticity-armed controller) and projected onto principal components computed across many bodies. The hope is to see clusters by body under the baseline collapsing to a shared trajectory under the rule — intuitively, "the rule makes all bodies look the same to the controller."

Status. Plots are being produced; results are still under interpretation. The eventual analysis target is a CCA comparison between baseline and adapted activations across bodies (research question 4).

3.5 Work Package 2 Outer Loop — Morphology Evolution

3.5.1 Experiment WP2-Outer.A — Outer multi-objective morphology search wrapping an inner rule search

Question. After Experiment WP2.A established that flat joint evolution does not work, can morphology evolution still be useful when it operates *only* on the morphology genome, with the rule sub-problem delegated to an inner CMA-ES per candidate body?

Setup. A two-level optimizer:

- **Outer level:** a multi-objective evolutionary algorithm over the 15-dimensional morphology genome. Operators are simulated-binary crossover and polynomial mutation, configured with standard defaults from the literature.
- **Inner level:** for each candidate body in the outer population, the rule-evolution inner loop runs for a fixed number of inner generations, treating that body as the (single) target morphology. The fitness returned to the outer level is a tuple of evaluation metrics — by default forest-corridor progress (maximized) and cost of transport (minimized), corresponding to the slogan "go as far as possible, as efficiently as possible."

The first slot of the initial outer population is optionally seeded with the canonical drone genome, so every run carries a known-good morphology through selection regardless of how the random initial bodies score.

Expected output. A Pareto front in (progress, $-$cost-of-transport) space at each outer generation, together with the plasticity rule recovered for each Pareto-optimal body. The interesting scientific question (research question 3) is whether the Pareto-optimal bodies converge to a recognizable family across multiple runs with different seeds: if they do, that family is the co-design-optimal body for the chosen objectives.

Status. Implementation complete; the first production runs are pending after the inner-loop body refresh period sweep (Experiment WP2.I) finishes.

3.6 What each visualization is supposed to show

- **Per-rollout overlay videos.** Trajectory, top-down map of the flight through the forest, per-component reward time series (essential, because the rule frequently improves one term and regresses another), heatmap of the last-layer weight changes, and a trace of the rule's per-step output. For accident rollouts, the event is marked on the timeline and a snapshot of the weight heatmap before and after the event is overlaid.
- **Per-generation training plots.** Fitness curves (mean, best, percentile bands) for the population, separately for each reward component, plus the moving baseline reference. A normalized-vs-baseline mode subtracts the baseline so that rule contribution is visible directly.
- **Inner-loop optimizer state plots.** Step size, eigenvalues of the covariance matrix, and axis ratio over generations. Used to diagnose pathologies like premature convergence (step size collapsing before fitness plateaus) and to confirm that the optimizer is exploring the rule space rather than chasing a single direction.
- **Per-run summary plots.** Per-trial reward-component breakdowns, trajectory overlays across the rollouts of an evaluation, and weight-change histograms.

3.7 What didn't work, condensed

- **Flat joint evolution of rules and morphology in one multi-objective genome.** Wrong optimizer for the rule subspace, high-dimensional crossover actively harmful, and the two selection signals blurred each other. Replaced by the nested architecture.
- **Several parallelization back-ends for PPO training** (multi-GPU dispatcher, parallel scenes across processes, sharded body pool). All under-performed the simpler single-process backend once the simulator's compilation cache was warm, and were significantly more brittle. The parallel-scene sweep was useful as a diagnostic; the runtime code was eventually removed.
- **Single shared simulator state containing many different bodies, for evaluation.** Numerical failure on one body silently corrupted every other body's next step. Replaced by independent simulator states, one per body. The same shared-state setup is still used during training, where automatic episode resets bound the failure mode.
- **Skipping the simulator step for terminated environments.** Branching cost greater than the saving; reverted same day.
- **Plain ABCD updates with the standard "decay toward zero".** The decay destroyed the pre-trained controller. Replaced by decay anchored to the pre-trained weight.
- **Deterministic evaluation of a controller that was trained stochastically.** Silently degraded the baseline measurement until the stochasticity setting was wired through.
- **Per-individual forests and commanded speeds in the inner loop.** Made fitness rankings noisy until forests and speeds were pinned per generation rather than per individual.

3.8 Design choices that stuck

- Asymmetric controller training: morphology-blind actor, morphology-aware critic, with independent recurrent memories. This is the precondition for stable wide-pool training and for downstream Hebbian adaptation being a scientifically meaningful contribution.
- Plasticity rule restricted to the last linear layer of the controller, with anchored decay (always on) and optional Oja-style coefficient. Zero rule initialization at the start of each rule-evolution run.
- Independent simulator states for multi-body inner-loop evaluation, optionally distributed across workers and GPUs.
- Weight reset to the pre-trained values at the start of every rollout and every generation, so each rollout begins from the same well-behaved controller and the rule is judged on what it does *from there*.
- Outer multi-objective morphology search wrapping an inner covariance-adapting rule search, rather than one flat joint genome.
- Frozen-config, frozen-seed, frozen-checkpoint reproducibility bundle written into every run folder, and per-repeat seed auto-increment in the cluster drivers, so that any reported number can be reproduced and any sweep is a one-line configuration change.
- Baseline period plus specialist reference in the inner loop: fitness gains are normalized against forest difficulty drift and reported as fraction of the generalist-to-specialist gap closed.
- Per-source noise toggles so the rule's contribution can be cleanly ablated against each noise channel independently.

4 Starting Literature Suggestions

- Bergonti, Fabio, Nava, Gabriele, Wüest, Valentin, Paolino, Antonello, L'Erario, Giuseppe, Pucci, Daniele, and Floreano, Dario. *Co-Design Optimisation of Morphing Topology and Control of Winged Drones.* ICRA, 2024. (Co-design framework jointly optimizing drone morphology and control.)
- Gupta, Aniruddha K., Savarese, Silvio, Ganguli, Surya, and Fei-Fei, Li. *Embodied Intelligence via Learning and Evolution.* Nature Communications, 2021. (Joint evolution of morphology and learning-based control via DERL.)
- Soltoggio, Andrea, Stanley, Kenneth O., and Risi, Sebastian. *Born to learn: the inspiration, progress, and future of evolved plastic artificial neural networks.* Neural Networks, 2018. (Core reference on evolved plasticity and adaptive neural systems.)
- Miconi, Thomas, Stanley, Kenneth O., and Clune, Jeff. *Differentiable plasticity: training plastic neural networks with backpropagation.* ICML, 2018. (Key work on trainable Hebbian plasticity.)
- Finn, Chelsea, Abbeel, Pieter, and Levine, Sergey. *Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks.* ICML, 2017. (Foundational meta-learning framework.)

- Raghu, Maithra, et al. *Rapid Learning or Feature Reuse? Towards Understanding the Effectiveness of MAML.* ICLR, 2020. (Introduces ANIL and analyzes representation reuse.)
- Mr Fuda, et many people. *Emergent Heterogeneous Swarm Control Through Hebbian Learning.* arXiv, 2025.

5 Further Readings

- Oja, E. *Oja learning rule.* Scholarpedia, 3(3):3612, 2008. doi: 10.4249/scholarpedia.3612. revision #91606.
- Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. *SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability.* In Advances in Neural Information Processing Systems, pages 6076–6085, 2017.
- Ari S Morcos, Maithra Raghu, and Samy Bengio. *Insights on representational similarity in neural networks with canonical correlation.* arXiv preprint arXiv:1806.05759, 2018.